PLC_4 (Line 4) — TIA Portal build steps (click-through) — REFERENCE PLC

This is the reference PLC — it matches the S7-1500 sim you already run. Build this one **first**, verify it, then library-ize its shared blocks so PLC_1/2/3/5/6 reuse them. Deep reference: `../_shared/blocks.md`, `docs/line4/instructions.md`.

At a glance

Line	line4 (spans **this PLC** + PLC_6 for crane 7B)
Cranes	**6** (downstream, picks up at TRVL-4A) + **7A** (entry, has LOAD bays)
Endpoint	`opc.tcp://192.168.0.114:4840`
OPC UA namespace	`3`
CPU	`6ES7511-1AK02` (S7-1500)
Data DBs to create	`Crane6Data`, `Crane7AData`
Sequencer FBs to create	`Crane6_Seq`, `Crane7A_Seq`
IOMap	`./IOMap.scl`
OB1 order	`IOMap` → `Crane6_Seq` → `Crane7A_Seq` (`./OB1.md`)

> **If your existing Line 4 sim project already has `typeTask`, `TravelTo`, `CraneSequencer`, `Line4Data`:** you're mostly done. To make it match the per-crane model, rename/duplicate `Line4Data` into `Crane6Data` + `Crane7AData` and duplicate the FB per crane (Steps 3–4). The single-`Line4Data` sim still works, but the app now addresses cranes by their own DB name.

Step 0 — Prerequisites

TIA V18 + PLCSIM Advanced V5.0 (version-locked; do not mix with V19/V6). Run PLCSIM **as Administrator**. Sim network: PLC `192.168.50.1`, Windows PLCSIM adapter `192.168.50.100`, endpoint `opc.tcp://192.168.50.1:4840` (the real target uses `192.168.0.114`). See `docs/tia_opcua/tia-plcsim-opcua-setup.md`.

Step 1 — Device + network + OPC UA server

1. Add CPU **`6ES7511-1AK02`**.
2. **Ethernet:** real IP **`192.168.0.114`** (sim uses `192.168.50.1`), mask `255.255.255.0`.
3. **OPC UA** → **Server** → **Activate** → `opc.tcp://…:4840`.
4. **Add the exact I/O modules from Panel_4** — not BA/16-ch: **two** `DI 32x24VDC HF` = `6ES7521-1BL00-0AB0` (→ `%I0–%I3` crane 6, `%I4–%I7` crane 7A) and one `DQ 32x24VDC/0.5A HF` = `6ES7522-1BL01-0AB0` (→ `%Q0–%Q3`). Confirm start bytes **0 / 4 / 0** cover the `./IOMap.scl` `%I`/`%Q`.

Step 2 — Library-ize the shared blocks (once — this is where the library comes from)

The origin project already exists: `CraneSim.ap18` (repo root). Open it — it already contains `typeTask` (UDT), `TravelTo` (FC), `CraneSequencer` (FB) and `Line4Data`. You do **not** create these by hand (that's only the fallback in `../_shared/blocks.md` if you ever start from nothing).

Library-ize from `CraneSim`: View → Libraries → **Global libraries** → **Create new global library** `CraneAutomation_Lib`; drag `typeTask`, `TravelTo`, `CraneSequencer` into **Master copies**; **Save library**. PLC_1/2/3/5/6 will drag these in instead of retyping. (Do **not** put `Line4Data` or `IOMap` in the library — those are per-PLC. You may keep `Line4Data` as a *template DB* master copy if you want a copy-per-crane starting point.)

Step 3 — Create the two data DBs (copy + rename)

Create DB	For crane
`Crane6Data`	6
`Crane7ADData`	7A

Each: Global DB, **Optimized ON**, rows per `../_shared/blocks.md §2`. Tick ***Accessible + Writable from HMI/OPC UA*** on `task_count`, `cmd_send`, `tasks[].station`, `tasks[].dwell_s` (`sts_*`/`status` = *Accessible* only). Compile.

Step 4 — Create the two sequencer FBs (copy + ONE find/replace)

Copy `CraneSequencer` → rename	Find/replace inside it
`Crane6_Seq`	`Line4Data` → `Crane6Data`
`Crane7A_Seq`	`Line4Data` → `Crane7ADData`

DB rename is the **only** edit. Compile each.

Step 5 — Create the `IOMap` FC (real wiring)

1. **Add new block** → **Function (FC)**, SCL, **Return Void**, name `IOMap`.
2. Paste `./IOMap.scl`; confirm `%I`/`%Q` match your modules.
3. **Travel direction (already wired in the file):** West = forward = right (`R-West ← mv_fwd`), East = reverse = left (`R-East ← mv_rev`). The crane advances to higher stations by moving West.

Open wiring TODO for PLC_4 (fill from the real drawings before running 7A on hardware; on the sim drive these bits from a watch table — crane 6 runs fully with no TODOs):

- **Crane 7A parking** `ls_at[0]` — no PRK switch in the panel extract.
- **Crane 7A per-tank switches** `ls_at[2..7]` — tanks `1-1A`(2), `1-1B`(3), `1-2`(4), `1-3`(5), `1-4A`(6), `1-4B`(7).

Crane 6 is fully wired (Loading = its TRVL-4A transfer-pickup switch, downstream crane).

Step 6 — Wire OB1

Main [OB1] (LAD): `IOMap` (Net 1) → `Crane6_Seq` (inst `Crane6_Seq_DB`) → `Crane7A_Seq` (inst `Crane7A_Seq_DB`). Parallel every scan. OB1 stays LAD/FBD (V18 can't make OB1 SCL); the SCL lives in the FC/FB. Detail incl. zero-latency-output variant: `./OB1.md`.

Step 7 — Compile, download, RUN

1. Project → Properties → Protection → tick **"Support simulation during block compilation"** (needed for PLCSIM Advanced).
2. **Compile** → **Software (rebuild all)** → 0 errors.
3. **Download** (fresh DB → in Load preview set **Stop modules = "Stop all"**, then **"Delete and replace system data"**). Set **Start module** → CPU **RUN**. `:4840` opens only in RUN.

Step 8 — Real PLC only: IP, security, cert, clock

1. Real IP `192.168.0.114`; confirm which NIC (`.114` vs `.214`) runs OPC UA.
2. Set `securityMode`/`securityPolicy` on the PLC_4 connection to match the secured endpoint.
3. Move our client cert to the PLC's **trusted** store.
4. **Fix the PLC clock.** If `ping` works but the app gets `ECONNREFUSED` on `:4840`, the OPC UA server didn't start (often a wrong CPU clock → stale cert): see `docs/tia_opcua/opcua-server-wont-start-econnrefused.md`.

Step 9 — Sim verification (proven flow)

Per `docs/line4/monitoring-watch-table.md`, per crane (`Crane6Data`, then `Crane7AData`):

1. Seed `tasks[1].station=12, dwell_s=6`, `task_count=1`; set `ls_at[0]=TRUE`, `ls_up=TRUE`.
2. Modify `cmd_send → TRUE`; flip `ls_at`/`ls_dip`/`ls_up`/`btn_start` to walk `sts_state` 10→20→30→40→50→60→70→80→85→90.
3. `cmd_send → FALSE` → `sts_done` clears, back to 0. Repeat on the other crane → both advance in parallel from one OB1.

App / PLC-Manager wiring (NodeIds) — verified live on this PLC

Namespace **3**. Bind PLC_4 → line4 cranes 6/7A, endpoint above.

```
ns=3;s="Crane6Data"."tasks"[0]."station"   Int   write   (OPC UA [0] = S7
tasks[1])
ns=3;s="Crane6Data"."tasks"[0]."dwell_s"   Int   write
ns=3;s="Crane6Data"."task_count"           Int   write
ns=3;s="Crane6Data"."cmd_send"             Bool  write
ns=3;s="Crane6Data"."sts_busy" / "sts_done" / "sts_step"   read
ns=3;s="Crane6Data"."tasks"[0]."status"    Int   read
```

Crane 7A identical with `Crane7AData`. **Two S7-1500 array quirks (both verified live):** the bracket goes **outside** the quotes (`"tasks"[0]."station"`), and arrays are **re-based to 0** over OPC UA (OPC UA `[0]` = S7 `tasks[1]`, so the app writes index `taskNo - 1`). See `docs/line4/instructions.md` Part 6.

Done when…

- [ ] `typeTask` + `Crane6Data` + `Crane7AData` compile; *Writable* set on app-written rows.
- [ ] `TravelTo` + `Crane6_Seq` + `Crane7A_Seq` compile; only edit was the DB rename.
- [ ] `IOMap` compiles; 7A TODOs filled or noted as sim-driven (crane 6 has none).
- [ ] OB1: `IOMap` → `Crane6_Seq` → `Crane7A_Seq`; rebuild-all = 0 errors; CPU RUN.
- [ ] Sim reaches 90 and back to 0 on both cranes; NodeIds verified live.
- [ ] `CraneAutomation_Lib` saved with the three shared blocks (so the other PLCs reuse them).
- [ ] Real PLC: security + cert + clock done; project saved as `PLC_4`.